

SIMATS ENGINEERING

ASSESSMENT TOOL 1 (High)- Analytical Problem Solving

COURSE NAME: Data Structures

COURSE CODE:CSA0316

COURSE OUTCOME COVERED

CO2 : Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques. (BTL3, BTL4)

Assessment Tool Weightage: **30%**

QUESTIONS:

- 20 Mr. XYZ booked a ticket on Pandian Express. He reached the station and was looking for his PNR number in the chart. Call out binary search procedure to search the PNR number 4056786. The ticket numbers are 4056779, 4056780, 4056781, 4056782, 4056783, 4056784, 4056785, 4056786, 4056787, 4056788, 4056789.
- 21 Convert the expression $((a + b) + c * (d + e) + f) * (g + h)$ to 1) Prefix expression , 2) Postfix expression.

Code of Conduct:

I _____M.Vineela(192472071)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Numerical Problems

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Question 20 :

Mr. XYZ booked a ticket on Pandian Express. He reached the station and was looking for his PNR number in the chart. Call out binary search procedure to search the PNR number 4056786. The ticket numbers are 4056779, 4056780, 4056781, 4056782, 4056783, 4056784, 4056785, 4056786, 4056787, 4056788, 4056789.

Program:

```
#include <stdio.h>

int main() {
    int a[] = {4056779, 4056780, 4056781, 4056782, 4056783,
               4056784, 4056785, 4056786, 4056787, 4056788, 4056789};
    int n = 11;
    int key = 4056786;
    int low = 0, high = n - 1, mid;
    int found = 0;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key) {
            printf("PNR number %d found at position %d\n", key, mid + 1);
            found = 1;
            break;
        }
        else if (key < a[mid]) {
            high = mid - 1;
        }
        else {

```

```

        low = mid + 1;
    }
}
if (!found)
    printf("PNR number not found\n");
return 0;
}

```

Output:

The screenshot displays the OnlineGDB web interface. On the left is a sidebar with navigation links: IDE, My Projects, Classroom (marked as new), Learn Programming, Programming Questions, Sign Up, and Login. The main area shows a C program in a dark-themed editor. The program defines an array 'a' with 11 elements, sets 'n' to 11, and searches for the key '4056786'. It uses a while loop for binary search, updating 'low' and 'high' pointers. When the key is found at index 8, it prints the message and returns 0. The output console at the bottom shows the execution result: 'PNR number 4056786 found at position 8', followed by a green message indicating the program finished with exit code 0 and a prompt to press ENTER to exit the console.

```

main.c
3- int a[] = {4056779, 4056780, 4056781, 4056782, 4056783,
4-           4056784, 4056785, 4056786, 4056787, 4056788, 4056789};
5- int n = 11;
6- int key = 4056786;
7- int low = 0, high = n - 1, mid;
8- int found = 0;
9- while (low <= high) {
10-     mid = (low + high) / 2;
11-
12-     if (a[mid] == key) {
13-         printf("PNR number %d found at position %d\n", key, mid + 1);
14-         found = 1;
15-         break;
16-     }
17-     else if (key < a[mid]) {
18-         high = mid - 1;
19-     }
20-     else {
21-         low = mid + 1;
22-     }
23- }
24-
25- if (!found)
26-     printf("PNR number not found\n");
27- return 0;
28- }

```

PNR number 4056786 found at position 8

...Program finished with exit code 0
Press ENTER to exit console.

Question 21:

Convert the expression $((a + b) + c * (d + e) + f) * (g + h)$ to 1) Prefix expression , 2) Postfix expression.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
char s[50];
int top = -1;
void push(char c) {
    s[++top] = c;
}
char pop() {
    return s[top--]; }
int p(char c) {
    if (c == '+' || c == '-') return 1;
    if (c == '*' || c == '/') return 2;
    return 0;
} void convert(char in[], char out[]) {
    int i, j = 0;
    char c;
    for (i = 0; in[i]; i++) {
        c = in[i];
        if (isdigit(c))
            out[j++] = c;
        else if (c == '(')
```

```

        push(c);
    else if (c == ')') {
        while (s[top] != '(') out[j++] = pop();
        pop();
    }
    else {
        while (top >= 0 && p(s[top]) >= p(c))
            out[j++] = pop();
        push(c);
    }
}

while (top >= 0)
    out[j++] = pop();
out[j] = '\0';
}

void reverse(char a[]) {
    int i, j;
    char t;
    for (i = 0, j = strlen(a)-1; i < j; i++, j--) {
        t = a[i];
        a[i] = a[j];
        a[j] = t;
    }
}

int main() {
    char in[] = "((a+b)+c*(d+e)+f)*(g+h)";
    char post[50], pre[50];

```

```

convert(in, post);

strcpy(pre, in);

reverse(pre);

for (int i = 0; pre[i]; i++) {

    if (pre[i] == '(') pre[i] = ')';

    else if (pre[i] == ')') pre[i] = '(';

}

top = -1;

convert(pre, pre);

reverse(pre);

printf("Prefix : %s\n", pre);

printf("Postfix: %s\n", post);

return 0;}

```

Output:

The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links: OnlineGDB, code. compile. run. debug. share., IDE, My Projects, Classroom, Learn Programming, Programming Questions, Sign Up, and Login. The main area displays a C program for infix to postfix conversion. The program uses a stack to convert the infix expression '++ab+c*def+gh' to its prefix and postfix forms. The output shows the prefix as '++ab+c*def+gh' and the postfix as 'ab+cde++f+gh+'. The program finishes with exit code 0.

```

main.c
42 int i, j;
43 char t;
44 for (i = 0, j = strlen(a)-1; i < j; i++, j--) {
45     t = a[i];
46     a[i] = a[j];
47     a[j] = t;
48 }
49 }
50 int main() {
51     char in[] = "((a+b)+c*(d+e)+f)*(g+h)";
52     char post[50], pre[50];
53     convert(in, post);
54     strcpy(pre, in);
55     reverse(pre);
56
57     for (int i = 0; pre[i]; i++) {
58         if (pre[i] == '(') pre[i] = ')';
59         else if (pre[i] == ')') pre[i] = '(';
60     }
61     top = -1;
62     convert(pre, pre);
63     reverse(pre);
64     printf("Prefix : %s\n", pre);
65     printf("Postfix: %s\n", post);
66     return 0;
67 }

```

Prefix : ++ab+c*def+gh
Postfix: ab+cde++f+gh++

...Program finished with exit code 0
Press ENTER to exit console.